Le type Liste : Définition

- Une liste est une collection ordonnée et **modifiable** (mutable) d'objets éventuellement hétérogènes séparés par des virgules et définis entre crochets [].

```
In [11]: animaux = ["girafe", "tigre", "singe", "souris"]

In [12]: tailles = [5, 2.5, 1.75, 0.15]

In [13]: mixte = ["girafe", 5, "souris", 0.15]

In [14]: animaux
Out[14]: ['girafe', 'tigre', 'singe', 'souris']

In [15]: tailles
Out[15]: [5, 2.5, 1.75, 0.15]

In [16]: mixte
Out[16]: ['girafe', 5, 'souris', 0.15]
```

```
In [18]: Listes=[animaux,tailles]

In [19]: Listes
Out[19]: [['girafe', 'tigre', 'singe', 'souris'], [5, 2.5, 1.75, 0.15]]
```

Le type Liste : Création d'une liste

Pour le moment:

- **définition des listes par les valeurs contenues** (*"en extension"*):

```
lst1 = [1,6,9,2,7,4,12,11,21,4]
```

- **construction de nouvelle liste par boucle for :**

```
# liste des carrés des impairs
lst2 = []
for v in lst1:
    if v%2==1 :
        lst2.append(v**2)
```

```
res = []
for v in itérable:
    if condition(v) :
        res.append(calcul(v))
```

Le type Liste : Création d'une liste

- En python, on peut construire des listes très facilement. Par exemple, si on veut construire la liste des carrés des nombres de 1 à n:

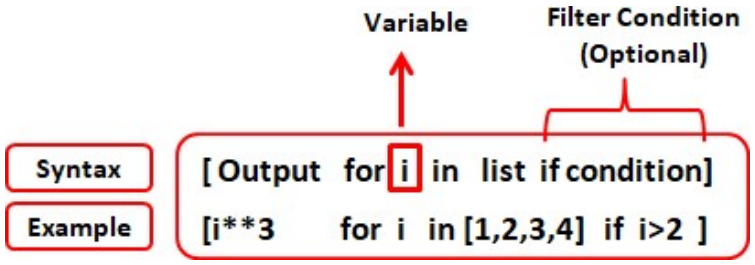
```
[ i**2 for i in range(1,n+1)]
```

```
In [101]: list2=[ i**2 for i in range(1,5)]

In [102]: list2
Out[102]: [1, 4, 9, 16]
```

- Si on veut uniquement les carrés des nombres multiples de 3 mais pas de 2 compris entre 1 et n :

```
[ i**2 for i in range(1,n+1) if i%3==0 and i%2!=0]
```




```
In [104]: list3=[ i**2 for i in range(1,10) if i%3==0 and i%2!=0]

In [105]: list3
Out[105]: [9, 81]
```

Le type Liste : Création d'une liste

Définition des listes par une expression
(en "*intension*" ou "*compréhension*")

À partir d'une séquence de valeurs, créer une nouvelle séquence...

```
res = [ calcul(v) for v in itérable ]
```

...avec un filtrage...

```
res = [ calcul(v) for v in itérable if condition(v) ]
```

...et on peut enchaîner les niveaux de boucles...

```
res = [ calcul(v1,v2)
          for v1 in itérable1
            for v2 in itérable2
              if condition(x) ]
```

Le type Liste : Indexation

- L'indexation permet d'accéder directement à un élément de la liste.

```
In [21]: animaux
Out[21]: ['girafe', 'tigre', 'singe', 'souris']

In [22]: animaux[0]
Out[22]: 'girafe'

In [23]: animaux[1]
Out[23]: 'tigre'

In [24]: animaux[3]
Out[24]: 'souris'
```

- Les indices d'une liste de n éléments commence à 0 et se termine à n-1
- **Un indice négatif** permet d'accéder aux éléments **à partir de la fin**.

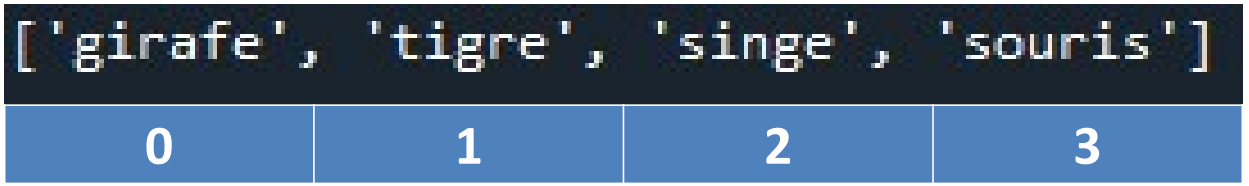
```
In [25]: animaux[4]
Traceback (most recent call last):

  File "C:\Users\Sawssen\AppData\Local\Temp/ipykernel_4628/2399852234.py", line 1, in
<module>
    animaux[4]

IndexError: list index out of range
```

Le type Liste : Slicing (découpage)

- Le slicing est une technique permettant d'accéder à un ensemble d'éléments dans une liste en indiquant un ensemble d'indexes.



Liste [début : fin : pas]

```
In [33]: animaux[0:3]
Out[33]: ['girafe', 'tigre', 'singe']
```

```
In [34]: animaux[2:]
Out[34]: ['singe', 'souris']
```

```
In [35]: animaux[:2]
Out[35]: ['girafe', 'tigre']
```

```
In [36]: animaux[::2]
Out[36]: ['girafe', 'singe']
```

Le type Liste : Slicing (découpage)

Soit S est une Liste :

- **S[i] :** retourne le (i+1)ème élément de S
- **S[:a]:** sous liste d'éléments de début jusqu'à l'élément d'indice **a** exclu
- **S[a:]:** sous liste d'éléments de l'indice **a** inclus jusqu'à la fin
- **S[:]:** toute la liste (la sous-liste complète)
- **S[a:b]:** sous liste d'éléments allant de l'indice **a** inclu jusqu'à l'élément d'indice **b** exclu
- **S[a:b:c]:** sous liste d'éléments de l'indice **a** inclu jusqu'à indice **b** exclu par pas égal à **c**

Le type Liste : les méthodes prédéfinies

```
In [54]: ListeNbr=[20,15,3,4,-1,0]

In [55]: ListeNbr.sort()#trier la liste en ordre croissant

In [56]: ListeNbr
Out[56]: [-1, 0, 3, 4, 15, 20]

In [57]: ListeNbr.sort(reverse=True)#trier la liste en ordre décroissant

In [58]: ListeNbr
Out[58]: [20, 15, 4, 3, 0, -1]

In [59]: ListeNbr.append(100) #ajouter 100 à la fin de la liste

In [60]: ListeNbr
Out[60]: [20, 15, 4, 3, 0, -1, 100]

In [61]: ListeNbr.reverse()#inverser la liste

In [62]: ListeNbr
Out[62]: [100, -1, 0, 3, 4, 15, 20]

In [63]: ListeNbr.remove(20)#supprimer la première occurrence de 20 de la liste

In [64]: ListeNbr
Out[64]: [100, -1, 0, 3, 4, 15]
```

Le type Liste : les méthodes prédéfinies

```
In [64]: ListeNbr
Out[64]: [100, -1, 0, 3, 4, 15]

In [65]: ListeNbr.index(3)#retourne l'indice de la première occurrence de 3 dans la liste
Out[65]: 3

In [66]: ListeNbr[1]=8# remettre l élement d indice 1 de la liste à 8

In [67]: ListeNbr
Out[67]: [100, 8, 0, 3, 4, 15]

In [68]: ListeNbr.pop()#supprime et renvoie le dernier élément de la liste
Out[68]: 15

In [69]: ListeNbr
Out[69]: [100, 8, 0, 3, 4]

In [70]: ListeNbr.extend([1,1,1])

In [71]: ListeNbr
Out[71]: [100, 8, 0, 3, 4, 1, 1, 1]

In [72]: ListeNbr.count(1)#retourne le nombre d occurrence de 1 dans la liste
Out[72]: 3
```

Le type Liste : Application 1

Jours de la semaine

Constituez une liste `semaine` contenant les 7 jours de la semaine.

1. À partir de cette liste, comment récupérez-vous seulement les 5 premiers jours de la semaine d'une part, et ceux du week-end d'autre part ? Utilisez pour cela l'indiçage.
2. Trouvez deux manières pour accéder au dernier jour de la semaine.
3. Inversez les jours de la semaine en une commande.

```
In [91]: semaine=['Lundi','Mardi','Mercredi','Jeudi','Vendredi','Samedi','Dimanche']

In [92]: print(semaine)
['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi', 'Samedi', 'Dimanche']

In [93]: semaine[0:5]
Out[93]: ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi']

In [94]: semaine[5:len(semaine)]
Out[94]: ['Samedi', 'Dimanche']
```

```
In [96]: semaine[-1]
Out[96]: 'Dimanche'

In [97]: semaine[len(semaine)-1]
Out[97]: 'Dimanche'
```

```
In [98]: semaine.reverse()

In [99]: print(semaine)
['Dimanche', 'Samedi', 'Vendredi', 'Jeudi', 'Mercredi', 'Mardi', 'Lundi']
```

Le type Liste : Application 2

Soit la liste : S = [10, 20, 50, 25, 30] :

1. Définir puis triez et affichez la liste S ;
2. ajoutez l'élément 12 à la liste et affichez la liste ;
3. renversez et affichez la liste ;
4. affichez l'indice de l'élément 25;
5. enlevez l'élément 50 et affichez la liste ;
6. affichez la sous-liste du 2ème au 3ème élément ;
7. affichez la sous-liste du début au 2ème élément ;
8. affichez la sous-liste du 3ème élément à la fin de la liste ;
9. affichez la sous-liste complète de la liste ;
10. affichez le dernier élément en utilisant un indexage négatif.

```
In [109]: S=[10,20,50,25,30]

In [110]: S.sort()

In [111]: print(S)
[10, 20, 25, 30, 50]

In [112]: S.append(12)

In [113]: S
Out[113]: [10, 20, 25, 30, 50, 12]

In [114]: S.reverse()

In [115]: S
Out[115]: [12, 50, 30, 25, 20, 10]

In [116]: S.index(25)
Out[116]: 3

In [117]: S.remove(50)

In [118]: S
Out[118]: [12, 30, 25, 20, 10]

In [124]: S[1:3]
Out[124]: [30, 25]

In [125]: S[:2]
Out[125]: [12, 30]

In [126]: S[2:]
Out[126]: [25, 20, 10]

In [127]: S[:]
Out[127]: [12, 30, 25, 20, 10]

In [128]: S[-1]
Out[128]: 10
```

Le type **tuple** : Définition

- Le tuple Python est l'implémentation **en lecture seule** de la liste: C'est- à-dire, il s'agit d'une collection ordonnée non modifiables **d'éléments éventuellement hétérogènes**. On parle ainsi de **liste immutable**.

- On ne peut donc pas modifier ou supprimer les éléments d'un tuple individuellement (on peut juste redéfinir un tuple dans sa totalité).

- Un tuple est un **objet itérable**.

```
t=(1,False)
print(t[0]) # 1
print(t[1]) # False
a, b=t# a = 1, b = False
```



Ce ne sont pas des tableaux : `t[0]=1` ne fonctionnera pas!

```
# On peut fabriquer un tuple à partir de n'importe quel itérable
t=tuple(range(1,5)) # (1,2,3,4)
```

Le type **tuple** : Définition

- Déclaration classique : avec des parenthèses
- Notation abrégée sans parenthèses
- Pour un tuple à un élément, la virgule est importante.
- **Exemple:** préciser que l'on parle d'un tuple à un élément et non de la valeur 12.

```
In [156]: tup1=(1,2,5)

In [157]: tup1
Out[157]: (1, 2, 5)

In [158]: tup2=5,8,1

In [159]: type(tup2)
Out[159]: tuple

In [160]: tup3=7,

In [161]: type(tup3)
Out[161]: tuple

In [162]: tup3
Out[162]: (7,)
```

Le type **tuple** : Définition

- Comme les listes, les tuples sont accessibles par indexation
- **in** permet de tester la présence d'un élément dans le tuple.
- Cependant, une fois créé le contenu d'un tuple ne peut être modifié.

```
In [167]: tup1
Out[167]: (1, 2, 5)

In [168]: tup1[0]
Out[168]: 1

In [169]: 2 in tup1
Out[169]: True

In [170]: 6 in tup1
Out[170]: False

In [171]: tup1[0]=9
Traceback (most recent call last):

  File "C:\Users\Sawssen\AppData\Local\Temp/ipykernel_4628/981832849.py", line 1, in <module>
    tup1[0]=9

TypeError: 'tuple' object does not support item assignment
```

Le type **tuple** : Définition

Remarque : critère de choix entre tuple et liste :

- Les tuples sont plus rapides à parcourir que les listes.
- Pour définir des constantes, utilisez des tuples
- il est possible de convertir des tuples vers des listes et réciproquement

```
In [172]: tup1
Out[172]: (1, 2, 5)

In [173]: list(tup1)
Out[173]: [1, 2, 5]

In [174]: tup5=tuple([1,True])

In [175]: tup5
Out[175]: (1, True)
```

Opérations communes (list, tuple, str)

Opération	Résultat
s[i]	élément d'indice i de s
s[i :j]	Tranche (slice) de i (inclus) à j (exclus)
s[i :j :k]	Tranche de i à (j-1) par pas de k
len(s)	Longueur de s
max(s), min(s)	max et min de s, si ces éléments sont comparables
x in s	True si x est dans s, False sinon
x not in s	True si x n'est pas dans s, False sinon
s+t	Concaténation de s et t
s*n, n*s	Concaténation de n copies de s
s.index(x)	Indice de la 1ère occurrence de x dans s
s.count(x)	Nombre d'occurrences de x dans s

où s et t sont des objets séquentiels de même type et i, j, k, n sont des entiers.